

Atelier 6

* * *

A06 - Vulnerable and Outdated Components

Introduction :

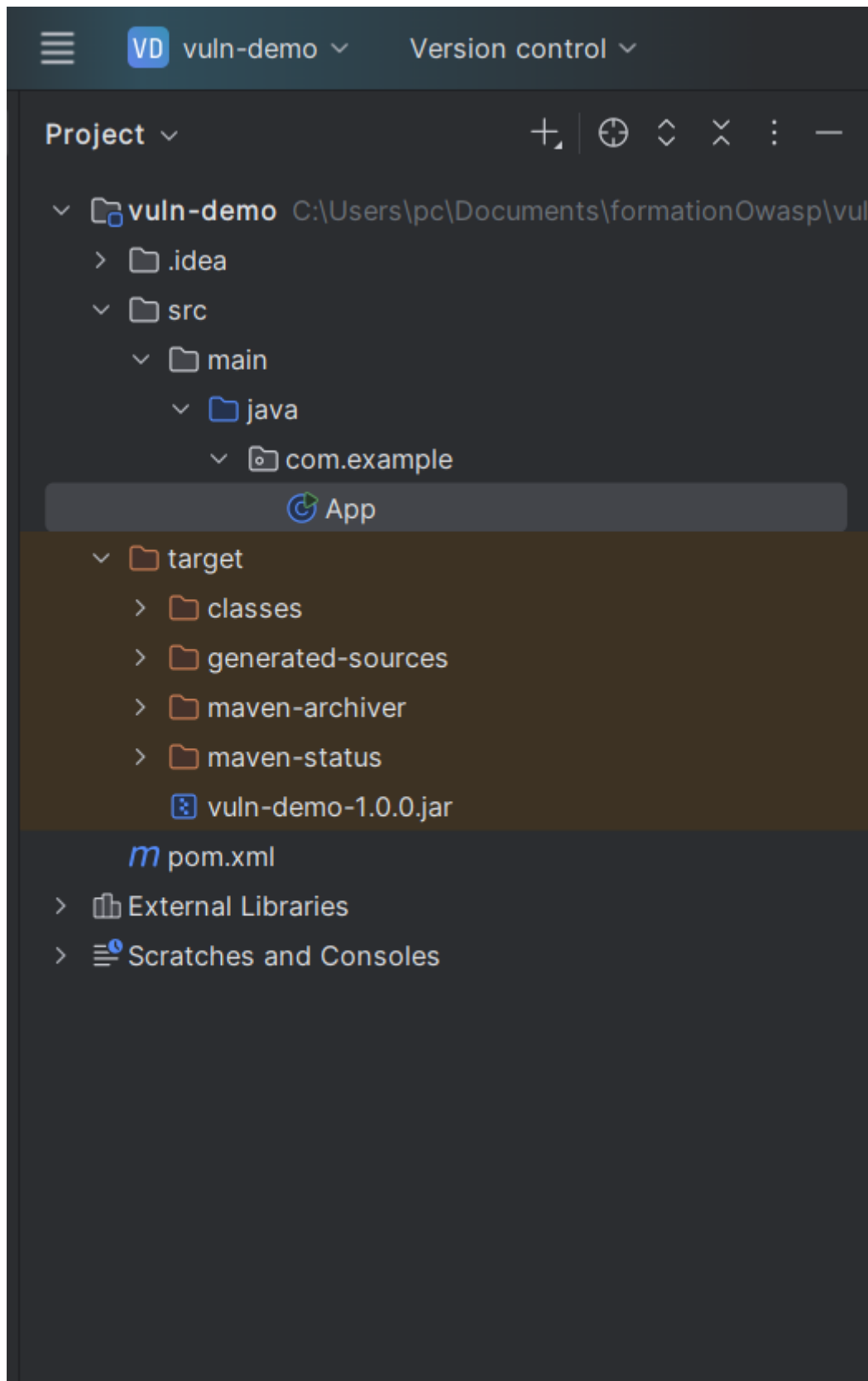
L'objectif de l'atelier est de comprendre comment une application peut être exposée lorsqu'elle utilise des composants obsolètes, ainsi que de mettre en pratique un scan de sécurité grâce à l'outil OWASP Dependency-Check. À travers un projet volontairement vulnérable (Log4j 2.14.1), nous avons pu identifier les failles connues, analyser leur impact et comprendre l'importance de maintenir les dépendances à jour.

1. Préparation du projet vulnérable

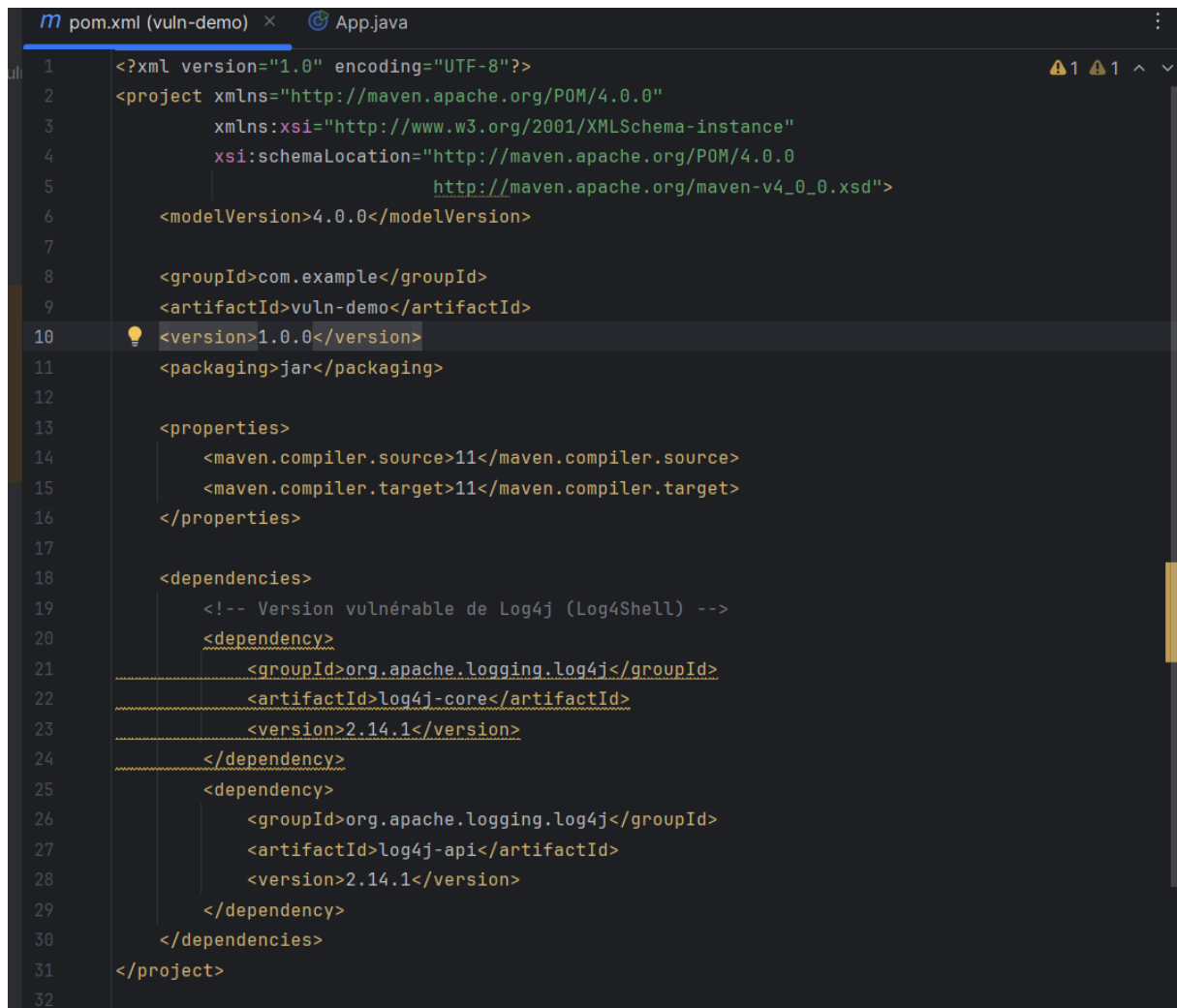
Créez un dossier local nommé : `.\formationOwasp\vuln-demo`

Dans ce dossier, crée la structure suivante :

```
vuln-demo/  
├─ pom.xml  
└─ src/main/java/com/example/App.java
```



Contenu du pom.xml : fichier contient volontairement une version vulnérable de Log4j.

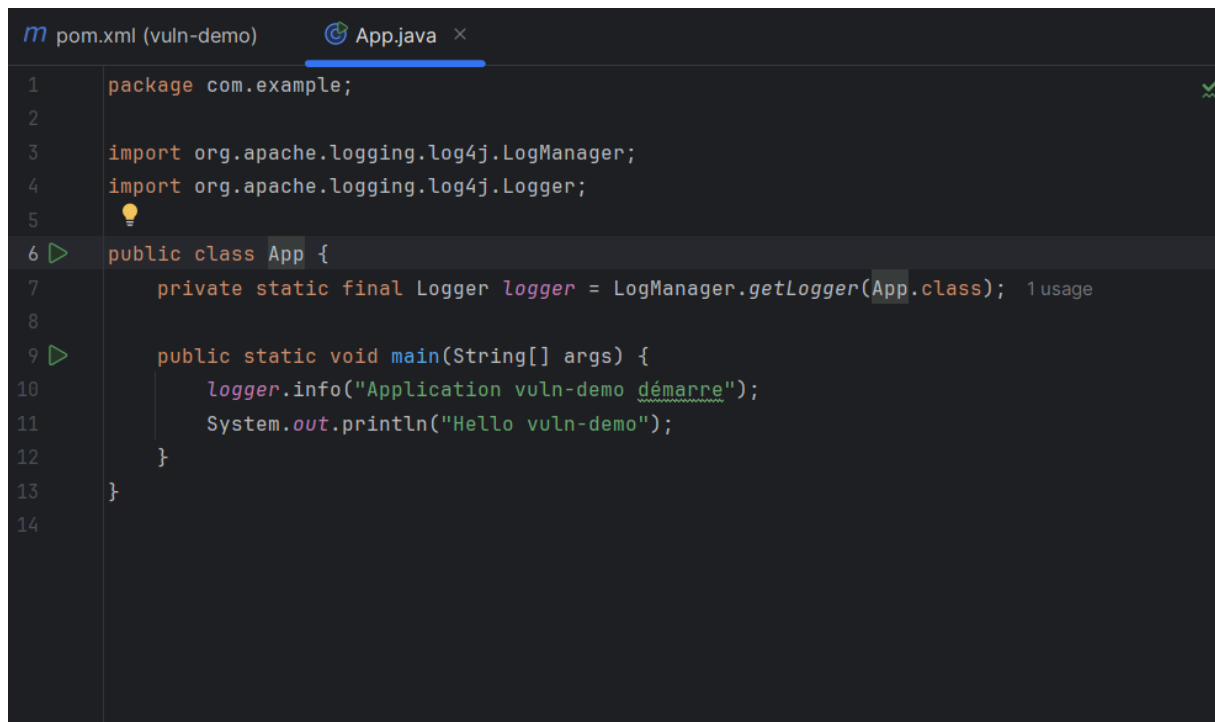


The image shows a code editor with two tabs: 'pom.xml (vuln-demo)' and 'App.java'. The 'pom.xml' tab is active, displaying the following XML content:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5                             http://maven.apache.org/maven-v4_0_0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7
8     <groupId>com.example</groupId>
9     <artifactId>vuln-demo</artifactId>
10    <version>1.0.0</version>
11    <packaging>jar</packaging>
12
13    <properties>
14        <maven.compiler.source>11</maven.compiler.source>
15        <maven.compiler.target>11</maven.compiler.target>
16    </properties>
17
18    <dependencies>
19        <!-- Version vulnérable de Log4j (Log4Shell) -->
20        <dependency>
21            <groupId>org.apache.logging.log4j</groupId>
22            <artifactId>log4j-core</artifactId>
23            <version>2.14.1</version>
24        </dependency>
25        <dependency>
26            <groupId>org.apache.logging.log4j</groupId>
27            <artifactId>log4j-api</artifactId>
28            <version>2.14.1</version>
29        </dependency>
30    </dependencies>
31 </project>
32
```

A lightbulb icon is visible next to line 10, indicating a suggestion or warning. The XML is well-formed, but the version of Log4j (2.14.1) is known to be vulnerable to Log4Shell.

Contenu du fichier App.java

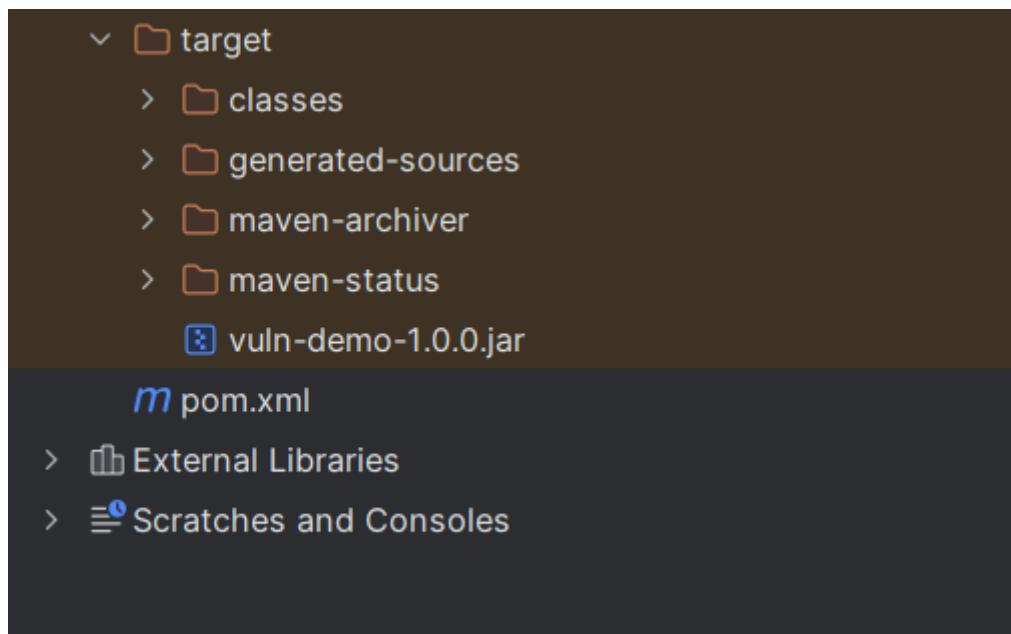


```
1 package com.example;
2
3 import org.apache.logging.log4j.LogManager;
4 import org.apache.logging.log4j.Logger;
5
6 public class App {
7     private static final Logger logger = LogManager.getLogger(App.class); 1 usage
8
9     public static void main(String[] args) {
10         logger.info("Application vuln-demo démarre");
11         System.out.println("Hello vuln-demo");
12     }
13 }
14
```

Compilation :

`mvn clean package`

Le jar sera créé dans `target/vuln-demo-1.0.0.jar`.



2. Scan avec d'OWASP Dependency-Check (version locale)

Télécharger et décompressez l'outil :

<https://github.com/jeremylong/DependencyCheck/releases>

bin	04/12/2025 11:23	Dossier de fichiers	
lib	04/12/2025 11:23	Dossier de fichiers	
licenses	04/12/2025 11:23	Dossier de fichiers	
plugins	16/02/2025 14:06	Dossier de fichiers	
LICENSE	16/02/2025 14:06	Document texte	12 Ko
NOTICE	16/02/2025 14:06	Document texte	1 Ko
README	16/02/2025 14:06	Fichier source Mar...	2 Ko

Commande complète pour lancer le scan et générer des rapports :

```
dependency-check.bat --project "vuln-demo" `
--scan "C:\D\formationOwasp\vuln-demo" `
--out "C:\D\formationOwasp\vuln-demo\odc-output" `
--format ALL
```

Si c'est la première fois, il faudra patienter pendant le téléchargement de la base NVD (CVE ~300 Mo). Les scans suivants seront beaucoup plus rapides.

```
PS C:\formationOWASP\vuln-demo> cd C:\dependency-check\bin
PS C:\dependency-check\bin> .\dependency-check.bat --project "vuln-demo" `
>> --scan "C:\formationOWASP\vuln-demo" `
>> --out "C:\formationOWASP\vuln-demo\odc-output" `
>> --format ALL
[INFO] Checking for updates
[WARN] An NVD API Key was not provided - it is highly recommended to use an NVD API key as the
PI Key
[INFO] NVD API has 320000000 records in this update
[INFO] Downloaded 1000000/320000000 (3%)
[INFO] Downloaded 2000000/320000000 (6%)
[INFO] Downloaded 3000000/320000000 (9%)
[INFO] Downloaded 4000000/320000000 (12%)
|
```

Une fois le scan terminé, le rapport :

C:\D\formationOwasp\vuln-demo\odc-output\dependency-check-report.html

← ↻ ⓘ Fichier C:/formationOWASP/vuln-demo/odc-output/dependency-check-report.html



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitute or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

♥ [Sponsor](#)

Project: vuln-demo

Scan Information ([show all](#)):

- *dependency-check version:* 12.1.0
- *Report Generated On:* Mon, 8 Dec 2025 22:02:00 +0100
- *Dependencies Scanned:* 1 (1 unique)
- *Vulnerable Dependencies:* 0
- *Vulnerabilities Found:* 0
- *Vulnerabilities Suppressed:* 0
- ...

Analysis Exceptions

Failed to request component-reports

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).
 This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).
 This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
 This report may contain data retrieved from [RetireJS](#).
 This report may contain data retrieved from the [Sonatype OSS Index](#).

Listera :

- Les dépendances détectées (log4j-core, log4j-api)
- Les vulnérabilités associées (CVE-2021-44228, CVE-2021-45046, etc.)
- Le score CVSS (niveau de gravité)
- Les recommandations (mettre à jour vers log4j $\geq 2.17.1$)

CONCLUSION :

L'atelier a permis de démontrer l'importance de surveiller et d'actualiser régulièrement les dépendances d'un projet. Le scan réalisé avec OWASP Dependency-Check a révélé plusieurs vulnérabilités critiques liées à Log4j, confirmant les risques liés aux composants obsolètes. Cette activité souligne la nécessité d'intégrer des outils automatisés de vérification dans le processus de développement afin de prévenir les failles et renforcer la sécurité globale des applications.